

ارائه‌های گروهی روز سوم مفاهیم پایه‌ای وب و شبکه

۱. DNS چیست و چگونه `google.com` به IP تبدیل می‌شود؟

DNS Domain Name System

سیستم نام دامنه مانند دفترچه تلفن اینترنت عمل می‌کند. انسان‌ها اسامی قابل حفظ مثل `google.com` را ترجیح می‌دهند، اما کامپیوترها برای ارتباط نیاز به آدرس‌های عددی (IP Address) دارند. DNS وظیفه دارد این نام‌ها را به IP معادل آن‌ها ترجمه کند.

فرآیند تبدیل `google.com` به IP (گام به گام):

بررسی حافظه‌ی پنهان (Cache):

مرورگر ابتدا حافظه‌ی خود، سپس سیستم‌عامل و در نهایت روتر خانگی را جستجو می‌کند. اگر اخیراً `google.com` بازدید شده باشد، آدرس IP در یکی از این کش‌ها پیدا می‌شود و کار تمام است.

Recursive DNS Resolver (حل‌کننده‌ی بازگشتی):

اگر IP در هیچ کشی نباشد، درخواست به یک DNS Resolver (معمولاً متعلق به ISP یا سرویس‌هایی مانند `8.8.8.8` گوگل) ارسال می‌شود. این سرور از طرف کلاینت فرآیند جستجو را کامل می‌کند.

Root Nameservers (سرورهای نام ریشه):

Resolver از یکی از ۱۳ سرور ریشه سؤال می‌کند: "مسئول دامنه‌های `com.` کیست؟" این سرورها اطلاعات دامنه‌های سطح بالا (TLD) را نگهداری می‌کنند و آدرس سرورهای `com` TLD را برمی‌گردانند.

TLD Nameservers

(سرورهای نام دامنه‌ی سطح بالا):

Resolver حالا از سرورهای TLD (مربوط به .com) می‌پرسد: "مسئول google.com کیست؟" این سرورها آدرس Authoritative Nameserver گوگل را دارند و آن را برمی‌گردانند.

Authoritative Nameserver (سرور نام اقتداری):

Resolver به سرور نام اختصاصی گوگل درخواست می‌دهد. این سرور رکورد نهایی (A Record) برای IPv4 یا AAAA برای IPv6 را که شامل آدرس IP سرور وب گوگل است برمی‌گرداند.

بازگشت پاسخ:

Resolver آدرس IP را دریافت می‌کند و آن را به مرورگر تحویل می‌دهد. مرورگر اکنون می‌تواند مستقیماً با آن IP ارتباط HTTP/HTTPS برقرار کند.

انواع رکوردهای مهم DNS:

A: نگاشت نام به IPv4

AAAA: نگاشت نام به IPv6

CNAME: نام مستعار برای یک دامنه (مثلاً www به دامنه اصلی)

MX: آدرس سرور ایمیل

TXT: اطلاعات متنی (مثل تأیید مالکیت دامنه)

اهمیت در بک‌اند:

DNS علاوه بر ترجمه‌ی نام، برای توزیع بار (Load Balancing) با برگرداندن IP‌های مختلف، بهبود کارایی با CDN و مسیریابی ترافیک به نزدیک‌ترین سرور استفاده می‌شود.

۲. Status Codes و HTTP Methods GET, POST, PUT, DELETE

HTTP Hypertext Transfer Protocol

پروتکل اصلی انتقال داده در وب است. هر درخواست از سمت کلاینت (مرورگر، اپلیکیشن) شامل یک Method (نوع عملیات درخواستی) می‌شود و سرور با یک Status Code (کد وضعیت) پاسخ می‌دهد.

HTTP Methods (افعال اصلی)

Method	کاربرد	ویژگی‌ها
GET	دریافت یک منبع	فقط خواندنی است، بدنه‌ی درخواست ندارد، پارامترها در URL، قابل کش شدن
POST	ایجاد یک منبع جدید	داده در بدنه‌ی درخواست ارسال می‌شود، معمولاً موجب تغییر در سرور می‌شود، قابل کش نیست
PUT	جایگزینی کامل یک منبع (یا ایجاد اگر وجود نداشته باشد)	Idempotent (چند بار اجرا نتیجه‌ی یکسان دارد)، داده‌ی کامل منبع را ارسال می‌کند
DELETE	حذف یک منبع	معمولاً Idempotent
PATCH	به‌روزرسانی جزئی یک منبع	فقط فیلدهای تغییر یافته ارسال می‌شود

HTTP Status Codes (کدهای وضعیت)

کدهای سه‌رقمی که نتیجه‌ی درخواست را نشان می‌دهند و به پنج دسته تقسیم می‌شوند:
xx1 (اطلاعاتی): درخواست دریافت شده و پردازش ادامه دارد.

xx2 (موفقیت):

- `200 OK` درخواست موفق (برای GET, PUT, PATCH)
- `201 Created` منبع جدید با موفقیت ایجاد شد (برای POST)
- `204 No Content` عملیات موفق اما بدنه‌ای در پاسخ نیست (برای DELETE)

xx3 (تغییر مسیر):

- `301 Moved Permanently` منبع برای همیشه جابجا شده است.
- `302 Found` تغییر مسیر موقت.
- `304 Not Modified` محتوا تغییر نکرده، از کش استفاده کن.

xx4 (خطای کلاینت):

- `400 Bad Request` درخواست نامفهوم است.
- `401 Unauthorized` نیاز به احراز هویت.
- `403 Forbidden` دسترسی ممنوع (حتی با احراز هویت).
- `404 Not Found` منبع پیدا نشد.
- `405 Method Not Allowed` متد درخواستی برای این endpoint مجاز نیست.
- `429 Too Many Requests` تعداد درخواست‌ها بیش از حد مجاز (Rate Limiting).

xx5 (خطای سرور):

- `500 Internal Server Error` خطای عمومی سرور.
- `502 Bad Gateway` سرور بالادستی پاسخ نامعتبر داده است.
- `503 Service Unavailable` سرور موقتاً در دسترس نیست.

چرا این مفاهیم در بک‌اند مهم‌اند؟

طراحی یک API RESTful دقیقاً بر پایه‌ی استفاده‌ی صحیح از این متدها و کدهای وضعیت است. این کار استانداردسازی ارتباطات، بهبود خوانایی و کاهش خطاهای سمت کلاینت را تضمین می‌کند.

۳. تفاوت HTTP و HTTPS SSL/TLS به زبان ساده

HTTP (نسخه‌ی ناامن)

داده‌ها را به صورت متن ساده بین کلاینت و سرور رد و بدل می‌کند. هر فردی که در مسیر شبکه قرار داشته باشد (مثلاً در یک شبکه‌ی عمومی وای‌فای) می‌تواند با ابزارهایی مانند Wireshark تمام اطلاعات شامل نام کاربری، رمز عبور و محتوای صفحات را بخواند. به این حمله‌ها ManintheMiddle گفته می‌شود.

HTTPS (نسخه‌ی امن)

همان HTTP است که روی یک لایه‌ی امنیتی به نام TLS (نسخه‌ی امروزی‌تر SSL) سوار می‌شود. این لایه سه کار اصلی انجام می‌دهد:

- محرمانگی (Encryption): داده‌ها رمزنگاری می‌شوند و فقط کلاینت و سرور می‌توانند آن‌ها را بخوانند.
- یکپارچگی (Integrity): از تغییر داده‌ها در مسیر جلوگیری می‌شود.
- احراز هویت (Authentication): سرور یک گواهی دیجیتال ارائه می‌دهد که هویت او را ثابت می‌کند.

دست‌دهی TLS Handshake فرآیند ساده‌شده:

- Client Hello: کلاینت به سرور می‌گوید "سلام، من این الگوریتم‌های رمزنگاری (cipher suites) را پشتیبانی می‌کنم."
- Server Hello: سرور یکی از آن‌ها را انتخاب می‌کند و گواهی SSL خود را که شامل کلید عمومی است می‌فرستد.
- اعتبارسنجی گواهی: کلاینت گواهی را با لیست مراجع صدور گواهی معتبر (Certificate Authorities) چک می‌کند.

- تبادل کلید: کلاینت یک کلید جلسه (session key) به صورت تصادفی ساخته، آن را با کلید عمومی سرور رمز می‌کند و می‌فرستد. فقط سرور با کلید خصوصی خود می‌تواند این پیام را رمزگشایی کند.
- ارتباط رمزنگاری‌شده: اکنون هر دو طرف کلید جلسه‌ی یکسان دارند و از این به بعد تمام داده‌ها با رمزنگاری متقارن (سریع) با آن کلید رد و بدل می‌شوند.

گواهی‌های SSL:

می‌توانید از سرویس‌های رایگان مانند Let's Encrypt برای دریافت گواهی استفاده کنید. مرورگرها سایت‌های دارای HTTPS را با علامت قفل نمایش می‌دهند و گوگل به آن‌ها رتبه‌ی بهتری در جستجو می‌دهد. امروزه تقریباً تمام API‌های حرفه‌ای فقط از HTTPS استفاده می‌کنند.

۴. ساختار JSON و XML (چرا دنیا رفت سمت JSON؟)

هر دو فرمت، روش‌هایی برای ذخیره و تبادل داده‌های ساختیافته هستند و هم برای انسان و هم برای ماشین خوانا طراحی شده‌اند.

JSON JavaScript Object Notation

- سبک وزن و بر اساس نحو زبان جاوااسکریپت.
- داده‌ها به صورت جفت‌های کلیدمقدار درون `{}` (شیء) یا لیستی از مقادیر درون `[]` (آرایه) ذخیره می‌شوند.
- انواع داده‌ی پشتیبانی‌شده: `string`، `number`، `boolean`، `null`، `object`، `array`.

XML eXtensible Markup Language

- ساختار درختی با تگ‌های باز و بسته (شبیه HTML).

- امکان تعریف ویژگی (attribute) درون تگ‌ها.
- قابلیت اعتبارسنجی قدرتمند با DTD یا XSD.

دلایل برتری و محبوبیت JSON

- حجم کمتر: JSON به دلیل نداشتن تگ‌های بسته‌شونده، ۲۰۳۰٪ حجم کمتری نسبت به XML دارد. مهم برای پهنای باند.
- خوانایی بهتر برای انسان: ساختار ساده‌تر، بدون قاطی شدن تگ‌ها و attribute ها.
- پردازش آسان در جاوااسکریپت: JSON مستقیماً با `JSON.parse()` به شیء قابل استفاده تبدیل می‌شود که دلیل اصلی محبوبیت آن در توسعه‌ی وب بود.
- پشتیبانی بومی انواع داده‌های ساده: تمایز بین عدد، رشته و بولین بدون نیاز به حدس زدن.
- هم‌راستایی با API های REST: REST به‌عنوان فرمت پیش‌فرض در RESTful API ها انتخاب شد و کتابخانه‌های تمام زبان‌ها از آن پشتیبانی می‌کنند.

آیا XML منسوخ شده است؟

خیر. XML هنوز در جاهایی که نیاز به ساختار پیچیده، فراداده (metadata)، یا اعتبارسنجی سخت‌گیرانه باشد (مانند پروتکل SOAP، فایل‌های پیکربندی Maven، فرمت‌های سند مثل DOCX) استفاده می‌شود. اما برای انتقال داده‌های ساده‌ی API، JSON انتخاب اول است.

۵. کلاینت در مقابل سرور (Frontend vs Backend)

معماری ClientServer

مدلی است که در آن یک برنامه (کلاینت) درخواستی را به برنامه‌ی دیگری (سرور) ارسال می‌کند و سرور پس از پردازش، پاسخی را برمی‌گرداند. این دو بخش به‌طور جداگانه توسعه می‌یابند و با پروتکل‌های استاندارد (معمولاً HTTP) با یکدیگر صحبت می‌کنند.

Frontend

- آنچه کاربر مستقیماً می‌بیند و با آن تعامل دارد.
- در وب: کدهای HTML، CSS و JavaScript که در مرورگر کاربر اجرا می‌شوند (React، Vue و ...).
- در موبایل: اپلیکیشن‌های iOS و Android.

وظایف:

- نمایش رابط کاربری (UI) و دریافت ورودی (کلیک، فرم)
- ارسال درخواست‌های HTTP به سرور (معمولاً AJAX با fetch یا axios)
- دریافت پاسخ (JSON) و به‌روزرسانی رابط کاربری
- اعتبارسنجی اولیه‌ی فرم‌ها (clientside validation)

Backend

- نرم‌افزاری که روی یک ماشین راه دور اجرا می‌شود و درخواست‌های کلاینت‌ها را پردازش می‌کند.
- تکنولوژی‌ها: پایتون (Django، FastAPI)، جاوا، Node.js و ...

وظایف:

- دریافت درخواست و مسیریابی (Routing)
- اعتبارسنجی نهایی داده‌ها (Serverside validation)
- پیاده‌سازی منطق کسب‌وکار (Business Logic)
- تعامل با پایگاه داده (PostgreSQL، MongoDB)
- احراز هویت و مجوزدهی
- ارسال پاسخ مناسب (JSON، HTML یا ...)

چرا این جدایی مهم است؟

- مقیاس‌پذیری مستقل: می‌توانید frontend و backend را روی سرورهای جداگانه و با تعداد نمونه‌های مختلف اجرا کنید.
- توسعه‌ی تیمی جداگانه: یک تیم روی frontend و تیم دیگر روی backend کار می‌کند بدون تداخل.
- امنیت: منطق حیاتی و دسترسی به دیتابیس هرگز در معرض دید کاربر قرار نمی‌گیرد.
- انعطاف‌پذیری: یک Backend می‌تواند به چندین Frontend مختلف (وب، اندروید، iOS) سرویس دهد. به این ترتیب شما API می‌سازید که هر کلاینتی بتواند از آن استفاده کند.

یک مثال عملی:

وقتی در سایت ثبت‌نام می‌کنید، فرم ثبت‌نام (Frontend) داده‌ها را با متد POST به آدرس `/api/register/` در سرور می‌فرستد. سرور (Backend) ایمیل تکراری را چک می‌کند، رمز عبور را هاش می‌کند و کاربر جدید را در دیتابیس ذخیره می‌کند. سپس پاسخ `Created 201` را به کلاینت برمی‌گرداند. کلاینت کاربر را به صفحه‌ی خوش‌آمدگویی هدایت می‌کند.